

SR4Rec: A reproducible toolkit for measuring whether super-resolution helps closed-set image recognition

Thai Le Quang^{a,b}, Vinh Truong Hoang^{a,*}

^a*Faculty of Information Technology, Ho Chi Minh City Open University, Ho Chi Minh City, Vietnam*

^b*Faculty of Data Science in Business, Ho Chi Minh University of Banking, Ho Chi Minh City, Vietnam*

Abstract

Super-resolution (SR) is often placed in front of recognition models that operate on low-resolution images, yet SR models are trained and ranked by image-fidelity metrics that do not say whether a recognizer becomes more accurate. SR4Rec is an open-source Python toolkit that answers this question for a user’s own dataset and SR models. From one configuration file and three commands, it checks the dataset, builds a closed-set split, produces low-resolution inputs, runs the bicubic baseline and every SR model, trains a recognizer for each image pipeline, and compares each SR model with bicubic using paired statistics over test images. The result is a self-contained report with Rank-1, Rank-5, CMC, macro precision, recall and F1, PSNR and SSIM, results by image size, CPU latency and before/after-SR panels, and a fingerprint of the code, data hashes and weights. On native ear images (D1, matched), real-world SR raised Rank-1 (Real-ESRGAN +1.4 pp; SwinIR real-world +1.1 pp) while bicubic-trained models reduced it; training the recognizer on bicubic images only reversed every verdict. On synthetic LFW faces (D2), no SR model changed Rank-1 significantly.

Keywords: super-resolution, image recognition, biometrics, evaluation protocol, reproducibility, PyTorch

*Corresponding author.

Email addresses: `thailq@hub.edu.vn` (Thai Le Quang), `vinh.th@ou.edu.vn` (Vinh Truong Hoang)

Required Metadata

Current code version

Nr.	Code metadata description	Value
C1	Current code version	v0.1.1
C2	Permanent link to code/repository used for this code version	https://github.com/quangthai121121/sr4rec/tree/v0.1.1
C3	Permanent link to Reproducible Capsule	https://doi.org/10.5281/zenodo.23011230
C4	Legal Code License	MIT
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	Python, PyTorch, timm, spandrel, NumPy, SciPy, statsmodels, scikit-learn, pandas, matplotlib, pydantic, Jinja2
C7	Compilation requirements, operating environments & dependencies	Python ≥ 3.10 ; PyTorch ≥ 2.3 ; Linux, Windows or macOS; CUDA or Apple GPU optional; dependencies in <code>pyproject.toml</code>
C8	If available Link to developer documentation/manual	https://github.com/quangthai121121/sr4rec#readme and docs/
C9	Support email for questions	thailq@hub.edu.vn

Table 1: Code metadata (mandatory).

1. Motivation and significance

Recognition from low-resolution images is common in practice: faces and ears captured far from the camera, animals photographed in the field, or objects cropped from wide-angle scenes. A frequent remedy is to upscale the input with a super-resolution (SR) model before recognition [1, 2, 3]. Modern SR models, from transformer-based restoration networks such as SwinIR [4] to models trained for real-world degradations such as Real-ESRGAN [5] and efficient networks such as SPAN [6], produce visually convincing images. They are, however, trained and ranked by image-fidelity measures such as PSNR and SSIM [7] on natural-image benchmarks. These measures do not tell whether a recognizer becomes more accurate: an SR model may sharpen edges while altering the fine structures that identify an individual, and published evidence on the downstream benefit of SR is mixed [1, 3, 8].

Table 2: Functionality available after installation.

	BasicSR	KAIR	timm	pyiqa	opensr-test	SR4Rec
Run pretrained SR models	✓	✓			✓	✓
PSNR / SSIM	✓	✓		✓	✓	✓
Train recognizers			✓			✓
Closed-set split with duplicate checks						✓
Recognizer trained per SR model (matched protocol)						✓
Paired statistics against bicubic						✓
Results by image size						✓
Native low-resolution images (no HR reference)						✓
Self-contained report and reproduction command						✓

Answering the question for a given dataset requires a controlled experiment, and such experiments are easy to get wrong. The recognizer must be trained and evaluated on a closed-set split in which every identity appears in every split and no image, not even a renamed copy, appears twice. The bicubic baseline and every SR model must be evaluated on the same test images with the same preprocessing. Evaluating an existing recognizer on SR outputs measures a domain shift as much as the effect of SR, so recognizers should also be trained on the images each SR model produces. Differences between SR models are often small, so they must be tested statistically, over test images and several training seeds, and reported separately for very small images, where SR matters most. Finally, the outcome depends on the degradation an SR model was trained for, which is rarely reported next to recognition results. In our experience each of these steps is re-implemented in ad-hoc scripts for every study, which makes results hard to compare and to reproduce.

Existing software covers parts of this workflow (Table 2). SR toolboxes such as BasicSR [9] and KAIR [10] train and test SR models and report PSNR and SSIM; spandrel [11] loads pretrained SR weights of many architectures; pyiqa [12] computes image-quality metrics; timm [13] provides recognition backbones; and opensr-test [14] benchmarks SR for remote sensing. None of them trains recognizers under a split and protocol designed for SR, tests the difference with bicubic, or reports the result by image size. SR4Rec is not a plug-in for an SR toolbox: SR toolboxes train SR models, whereas SR4Rec treats SR models as fixed inputs and controls the recognition experiment. Every component could be assembled from these libraries; the contribution of SR4Rec is the validated workflow that assembles them and the rules it enforces.

SR4Rec does not propose a new SR method. It provides a measurement instrument with four objectives, which structure the rest of this article: (O1) a single, validated protocol for the question “does this SR model help this recognizer on these data, compared with bicubic?”; (O2) any SR model, whether a weights file, a PyTorch class or images produced by another tool; (O3) conclusions stated with statistical support and by image size, including negative results; and (O4) results that anyone can regenerate with one command.

2. Software description

SR4Rec is a Python package built on PyTorch [15]. Users interact with it through a configuration file, `sr4rec.yaml`, and three commands: `sr4rec init` checks a dataset and writes a fully commented configuration, `sr4rec run` executes the workflow, and `sr4rec reproduce` re-runs a published demonstration and compares the results with the expected values within stated tolerances.

2.1. Software architecture

Figure 1 shows the architecture. The command-line interface and the Python API both construct a single `Run` object, so they produce identical results. `Run` calls six modules in a fixed order. *data* validates the dataset and creates or reads the split; *lr* produces low-resolution (LR) images; *sr* runs the bicubic baseline and every SR model; *rec* prepares the recognizer input and trains and evaluates the recognizers; *eval* and *stats* compute the metrics and the paired tests; and *report* writes the outputs. The configuration schema is defined once, with pydantic, and the same definition validates user files, generates the commented file written by `init`, and generates the configuration reference in the documentation, so the three cannot diverge. Unknown keys and invalid values are rejected with the list of accepted values.

Every expensive intermediate result (LR images, SR outputs, splits and trained recognizers) is cached under a key derived from the content it depends on: SHA-256 hashes of the images and of the SR weights or code, the configuration of the step, and the split. Re-running a study is therefore fast, adding an SR model trains only the recognizers of that model, and a stale result cannot be reused by mistake. Each run also writes `fingerprint.yaml` with the software and hardware environment and the hashes of the data and of every weight file.

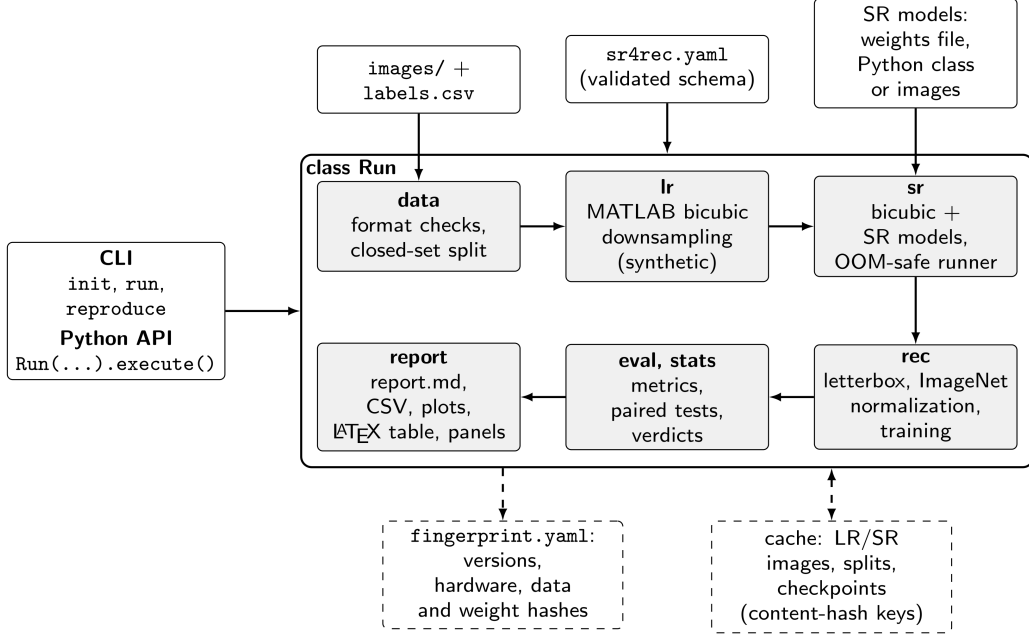


Figure 1: Architecture of SR4Rec. The CLI and the Python API share one `Run` class; expensive steps are cached under content-hash keys; every run writes a report and a fingerprint.

2.2. Software functionalities

Data and split. A dataset is a folder with `images/` and a `labels.csv` file listing the relative path, the label and, optionally, the split of every image. SR4Rec rejects missing or unreadable files, duplicate paths and classes that are too small, reporting each problem with its line number. The split comes either from the file or from user-given ratios applied within every class. Identical images (by SHA-256) are always kept in the same split, and a user-given split that places identical images in two splits is rejected.

Two data modes. In *synthetic* mode the images are high resolution (HR); SR4Rec crops them to a multiple of the scale factor and downsamples them with a MATLAB-compatible bicubic kernel, so PSNR, SSIM and an HR upper bound can be reported. In *native-lr* mode the images are already small, which is the realistic case for biometrics in the wild, and no downsampling or fidelity metric is applied.

Three SR sources. An SR model can be given as a weights file whose architecture spandrel detects, as a Python file with a PyTorch class that follows a four-rule interface (Section 2.3), or as a folder of images produced by any other tool. SR4Rec checks the scale and the output size of every model before any training (`run --dry-run`), pads inputs that are smaller than an architecture requires, and supports tiled inference. The report states the degradation each validated model was trained for, because a model trained for real-world degradation is not expected to excel on clean bicubic inputs.

Recognizer input and protocols. Every image, whether bicubic, SR output or HR, is letterboxed to a square without changing its aspect ratio, padded with the ImageNet mean colour and normalized with ImageNet statistics [16]. Under the default *matched* protocol, one recognizer is trained per image pipeline, backbone and seed, on the train images of that pipeline, and selected on its validation images; the test images are never used before evaluation. The *fixed_recognizer* protocol trains one recognizer on HR (or bicubic) images and applies SR at test time only; it is cheaper and answers a different question, which the report states. Backbones come from timm; ResNet-18 [17], MobileNetV3-Small [18] and ConvNeXt-Tiny [19] are validated. The training recipe is fixed so that SR models are compared under identical conditions.

Metrics and statistics. SR4Rec reports Rank-1 accuracy (the primary metric), Rank-5, the CMC curve, macro precision, recall and F1, and, in synthetic mode, PSNR and SSIM on the Y channel. For every backbone and SR model, per-image Rank-1 correctness is averaged over seeds and compared with bicubic on the same images: the difference is given with a 95% percentile bootstrap interval [20] and a paired sign-flip permutation test [21], Holm-corrected [22] over the SR models of each backbone. A fixed rule turns these into a verdict: significant gain, significant harm or no difference. The same analysis is repeated in four bins of LR image size.

Outputs and reproduction. A run folder contains `report.md`, which answers the question in at most three rule-generated sentences followed by the tables and figures, the CSV files behind every number, a L^AT_EX table, before/after-SR panels for selected test images, and the fingerprint. `sr4rec reproduce` runs a published demonstration either in full or, with `--eval-only`, from published recognizer checkpoints, and exits with a distinct code when a metric leaves its tolerance.

Robust execution. SR4Rec uses a CUDA GPU if available, otherwise an Apple GPU, otherwise the CPU. Running out of GPU memory never stops a run: SR switches to smaller tiles, training to gradient accumulation with the same effective batch size, evaluation to smaller batches, and, as a last resort, the step continues on the CPU. Each fallback is listed in the report.

2.3. Sample code snippet

Listing 1 shows the complete interface for adding an SR model as Python code; the model is then declared with one line in the configuration (`module: sr_models/tiny_espcn.py:TinyESPCN`).

Listing 1: An SR model added as a Python class (method 2). SR4Rec verifies the class, the scale and the output shape before use.

```
class TinyESPCN(torch.nn.Module):
    scale = 4                                # upscaling factor, checked by SR4Rec

    def __init__(self, weights=None):         # optional weights file
        super().__init__()
        self.body = torch.nn.Sequential(
            torch.nn.Conv2d(3, 32, 5, padding=2), torch.nn.ReLU(),
            torch.nn.Conv2d(32, 3 * self.scale ** 2, 3, padding=1),
            torch.nn.PixelShuffle(self.scale))
        if weights is not None:
            self.load_state_dict(torch.load(weights, weights_only=True))

    def forward(self, x):                     # (N,3,H,W) in [0,1] -> (N,3,4H,4W)
        return self.body(x)
```

2.4. Technical limitations

Version 0.1 addresses closed-set identification only; verification and open-set protocols, and attribute classification, are not included. Synthetic degradation is limited to bicubic downsampling with one scale factor per run. The training recipe is fixed by design, so that all SR models are compared under identical conditions; per-model tuning is not attempted. CPU latency is not measured for SR models given as images. Runs use one device.

3. Illustrative examples

The numbers in this section are produced by the demonstration configurations shipped with SR4Rec, frozen with tolerances in the package, and turned into the tables and figures below by `scripts/make_paper_assets.py`.

Table 3: D1: EarVN1.0, native low-resolution images, protocol matched, ResNet-18. Mean and sample standard deviation over seeds 0, 1, 2; Δ , CI and p compare each SR model with bicubic on the same 5,678 test images (paired bootstrap and sign-flip permutation test, Holm-corrected).

SR	Rank-1 (%)	Rank-5 (%)	Macro F1 (%)	Δ (pp)	95% CI (pp)	p (Holm)	Same sign	Verdict
bicubic (baseline)	77.9 $\pm$ 0.3	93.2 $\pm$ 0.2	77.8 $\pm$ 0.4					
realesrgan	79.3 $\pm$ 0.3	93.5 $\pm$ 0.1	79.1 $\pm$ 0.3	+1.4	[+0.7, +2.0]	0.0018	3/3	significant gain
swinir_classical	76.3 $\pm$ 0.4	92.4 $\pm$ 0.4	75.9 $\pm$ 0.3	-1.6	[-2.3, -1.0]	<0.001	3/3	significant harm
swinir_realworld	79.0 $\pm$ 0.5	93.2 $\pm$ 0.2	78.8 $\pm$ 0.6	+1.1	[+0.4, +1.8]	0.0044	3/3	significant gain
span	77.0 $\pm$ 0.5	92.6 $\pm$ 0.1	76.7 $\pm$ 0.4	-0.9	[-1.5, -0.3]	0.0044	3/3	significant harm

Quickstart. SR4Rec installs with `pip install -e .` from a repository clone. `sr4rec reproduce quickstart` writes the demo configuration because the quickstart expected results are not frozen in this development version. `sr4rec run` then executes that configuration on a small subset of Oxford-IIIT Pet [23] with an SR model given as Python code, on the reference machine, in under one minute. Appendix B shows the console session (`init`, `run --dry-run`, `run` and `reproduce`); the README walks that path step by step.

Experimental setup. All demonstrations compare bicubic with four validated SR models at scale 4: SwinIR classical [4] and SPAN [6], trained for bicubic degradation, and Real-ESRGAN [5] and SwinIR real-world, trained for real-world degradation [24]. The two SwinIR models share one architecture, so their difference isolates the training degradation. Every run uses the same untuned recipe: an ImageNet-pretrained ResNet-18 [17] fine-tuned for 30 epochs, input 224×224 , protocol matched. D1 uses EarVN1.0 [25] at its native resolution (164 identities, 5,678 test images); D2 uses LFW [26], people with at least 20 images, downsampled by 4 (62 identities); D1 and D2 run seeds 0, 1 and 2. D3 is a single run on CUB-200-2011 [27] with its official split and carries no comparative claim. The D1 configuration is listed in full in Appendix A.

Does SR help on natively small images? (D1). Under the matched protocol, Real-ESRGAN and SwinIR real-world improved Rank-1 on the same 5,678 test images (Table 3: +1.4 and +1.1 pp; Holm $p = 0.0018$ and 0.0044; same sign on all three seeds). SwinIR classical and SPAN reduced Rank-1 (-1.6 and -0.9 pp). The two SwinIR models share one architecture, so the sign follows the training degradation, not the network. The best matched result is still only +1.4 pp.

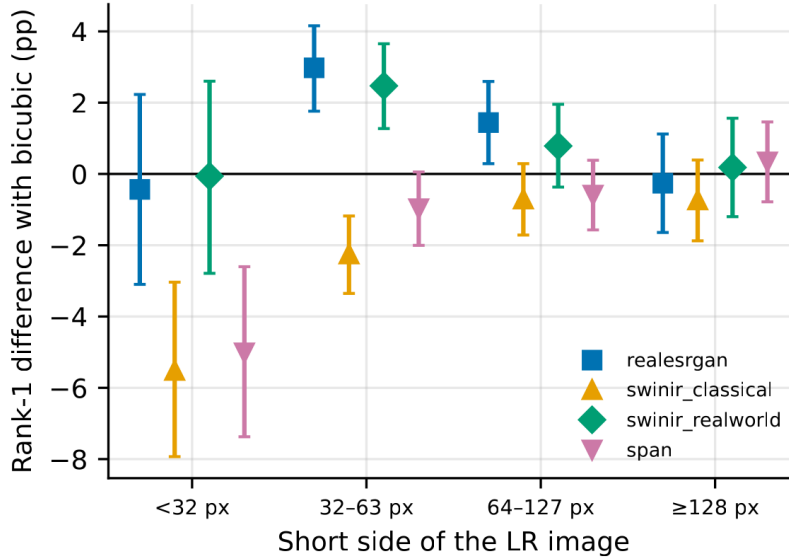


Figure 2: D1: Rank-1 difference with bicubic by short side of the LR image, with 95% bootstrap intervals over test images; hollow markers are bins with fewer than 50 test images (not tested).

Where it helps (D1 by image size). The gain of the real-world models is concentrated on mid-small ears (short side 32–63 px: Real-ESRGAN +3.0 pp, SwinIR real-world +2.5 pp; Figure 2, Table C.2). On the smallest bin (< 32 px) the classical models harm Rank-1 (SwinIR classical −5.5 pp, SPAN −5.0 pp) and the real-world models are indistinguishable from bicubic. Bins of 64 px and above show no significant difference.

What the protocol changes. Running D1 with the `fixed_recognizer` protocol, where one recognizer is trained on bicubic images and SR is applied at test time only, turns every significant gain into significant harm (Table 5: even the least-harmed model, SPAN, falls to −4.2 pp). The recognizer then sees SR outputs outside its training domain, while bicubic stays in domain.

Does image fidelity predict recognition? (D2). PSNR and SSIM rank the models in the same order as Rank-1 (Table 4, Figure 3: SwinIR classical 35.66 dB / Rank-1 91.1%; SPAN 35.56 dB / 90.9%; the real-world pair below bicubic on both). No difference with bicubic is significant (best +0.8 pp). Fidelity therefore does not tell whether an SR model helps recognition; the

Table 4: D2: LFW (people with at least 20 images), synthetic $\times 4$, ResNet-18, seeds 0, 1, 2. PSNR and SSIM on the Y channel of the test images; Rank-1 in %.

SR	Trained for	PSNR (dB)	SSIM	Rank-1 (%)	Δ (pp)	Verdict
bicubic (baseline)	n/a	30.39	0.899	90.4 ± 2.3		
realesrgan	real-world degradation	28.09	0.867	89.5 ± 1.2	-0.9	no difference
swinir_classical	bicubic degradation	35.66	0.951	91.1 ± 1.4	+0.8	no difference
swinir_realworld	real-world degradation	29.12	0.883	90.0 ± 1.2	-0.3	no difference
span	bicubic degradation	35.56	0.949	90.9 ± 1.8	+0.6	no difference
HR (upper bound)	n/a	-	-	92.9 ± 1.7		

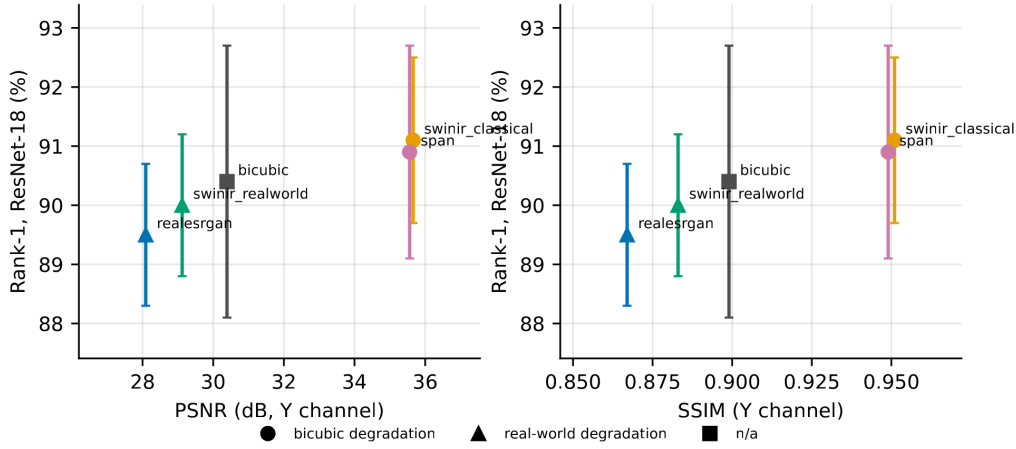


Figure 3: D2: PSNR and SSIM of each SR model against Rank-1 (mean $\pm$ sd over seeds); marker shape gives the degradation the model was trained for.

SwinIR pair again isolates training degradation on a controlled bicubic input. No face image is shown in this article.

Across domains. Table 5 collects the demonstrations. D3 is a single seed on CUB-200-2011 (official split): SPAN +1.2pp on that run, labelled ‘significant gain’ by the same per-image test as D1 and D2. With one seed, that label cannot be checked for agreement across independent training runs, so D3 shows that the same command works on another domain, not that SPAN reliably helps on CUB-200-2011.

Qualitative results. Figure 4 shows D1 test images for which an SR model corrected (rows one and three) or broke (rows two and four) the bicubic prediction, selected by a fixed rule rather than by hand. Across the test set, 793 images are corrected and 993 degraded by at least one SR model.

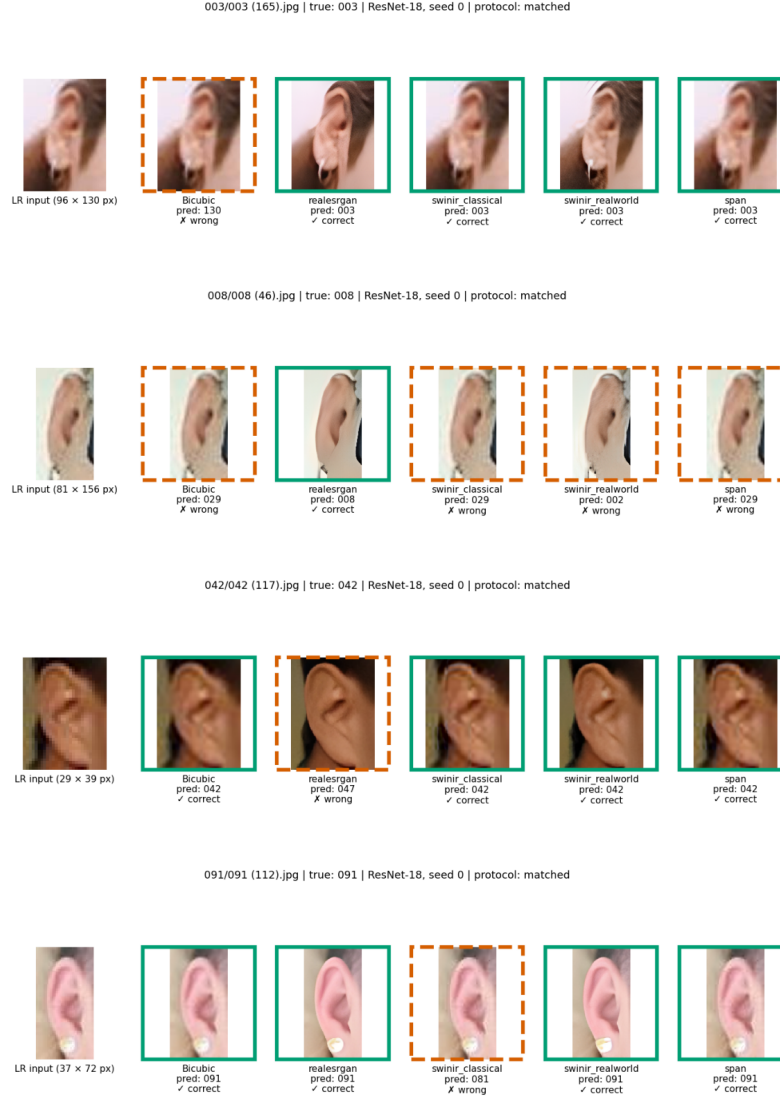


Figure 4: D1: LR input, bicubic, the four SR models; solid green frame: correct identity; dashed orange frame: wrong identity. Generated by `scripts/make_qualitative_figure.py`, the drawing function of the comparisons/panels of every report.

Table 5: The demonstrations: best SR model, its Rank-1 difference with bicubic and the verdict. The runs vary dataset, mode and protocol together, so they illustrate the workflow across domains rather than isolate one factor. Hours: wall-clock time of the whole run on the reference machine.

Demo	Dataset	Mode	Protocol	Seeds	Classes	Test	Best SR	Δ (pp)	Verdict	Hours
D1	earvn	native-lr	matched	3	164	5,678	realesrgan	+1.4	significant gain	13.8
D1, fixed recognizer	earvn	native-lr	fixed_recognizer	3	164	5,678	span	-4.2	significant harm	0.1
D2	lfw	synthetic	matched	3	62	602	swinir_classical	+0.8	no difference	0.8
D3	cub	synthetic	matched	1	200	5,794	span	+1.2	significant gain	4.0

Table 6: CPU latency in ms (median / p95): batch 1, LR input 64×64 , 4 threads, 13th Gen Intel(R) Core(TM) i7-13700F, 10 warm-up and 100 timed runs. Recognizer alone: ResNet-18 13.5 / 13.9.

	SR	SR only	SR + ResNet-18
bicubic		1.1 / 1.2	15.7 / 16.6
realesrgan		357.4 / 359.8	370.0 / 373.7
swinir_classical		501.3 / 505.3	514.0 / 597.1
swinir_realworld		537.5 / 630.7	551.4 / 639.0
span		18.9 / 19.2	31.7 / 32.3

Deployment. On CPU, SPAN adds 19ms to a 14ms ResNet-18 forward pass; Real-ESRGAN and the two SwinIR models add 350–540ms at the median (Table 6). SwinIR real-world shows the widest median-to-p95 gap (537–631ms alone; 551–639ms with the recognizer), and SwinIR classical widens the same way once combined with the recognizer (514–597ms); the other models stay close to their median. All GPU results come from one NVIDIA GeForce RTX 3080.

Reproduction. Reproduction is offered at two levels. `sr4rec reproduce d1_earvn --eval-only`, run against the released recognizer checkpoints, recomputes the SR outputs and the evaluation in 4.3 hours and matches the frozen values to two decimal places of Rank-1 (percentage points). Without the flag the command retrains the demonstration, 13.8 hours on the reference machine, and returns values within the frozen tolerance. Both exit with a distinct code when a metric drifts.

4. Impact

SR4Rec is intended for researchers who place super-resolution in front of a closed-set recognizer, and for groups that repeat that experiment on a new dataset or a new SR checkpoint.

It supplies a fixed protocol for a question that is usually answered with one-off scripts: does this SR model change Rank-1 relative to bicubic on these identities? The package checks the dataset, builds a closed-set split, runs bicubic and each SR model, trains a recognizer when the matched protocol requires it, and reports paired tests together with a breakdown by image size. Section 3 shows why that protocol is the product, not a side effect. On native ear images the sign of the Rank-1 difference followed the degradation the SR model was trained for and reversed when the recognizer was trained only on bicubic images. On synthetic LFW faces, PSNR and SSIM ranked the models but no SR model changed Rank-1 significantly. A study that reports only fidelity, or that applies SR only at test time, can therefore publish the opposite conclusion. SR4Rec makes those two checks part of the same run instead of a second codebase.

The library is also a base that other recognition papers can reuse. SR models are inputs: a weights file, a short PyTorch class, or a folder of images. A group can therefore keep its own restorer and still use the split, the statistics and the report. The command-line interface and the Python API share one `Run` object; expensive steps are cached by content hash; every run writes a fingerprint. That cuts the usual rewrite of dataset checks, split code and tables when the next dataset arrives. Published demonstrations can be compared with frozen expected values; rules for adding a validated SR model are in the repository documentation. The four demonstrations in this article took 18.7 GPU-hours on one NVIDIA GeForce RTX 3080.

5. Conclusions

This paper described SR4Rec, an open-source Python toolkit that compares given SR models with bicubic on a closed-set recognition split. Users provide a dataset and SR models; the commands `init`, `run` and `reproduce` produce the split, the metrics and a report. The software does not train a new SR network. On native ear images the effect of SR depended on the training degradation and on the evaluation protocol; on synthetic LFW faces no SR model changed Rank-1 significantly. Later versions are planned to cover verification and open-set recognition, which version 0.1 does not include.

CRediT authorship contribution statement

Thai Le Quang: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Writing – original draft, Visualization.

Vinh Truong Hoang: Conceptualization, Supervision, Writing – review & editing, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The datasets used in the demonstrations are public and must be obtained from their original sources; SR4Rec provides scripts that convert them. The archived code, expected results and recognizer checkpoints are available at <https://doi.org/10.5281/zenodo.23011230>.

Acknowledgements

The authors thank Ho Chi Minh City Open University and Ho Chi Minh University of Banking for institutional support.

Appendix A. Configuration of demonstration D1

Listing 2 is `src/sr4rec/demos/d1_earvn.yaml` as shipped.

Listing 2: Demonstration D1 configuration (`d1_earvn.yaml`).

```
# SR4Rec demo definition: d1_earvn
demo: d1_earvn
description: 'D1 (main): EarVN1.0 ears, native-lr x4, matched, bicubic + 4 SR, ResNet-18,
3 seeds'
defaults:
  data_root: data
  weights_root: weights
notes: python scripts/prepare_earvn.py --earvn <path_to_EarVN1.0> --out data/earvn. Put
the four validated SR weight
files in the weights folder (README, 'Validated SR models').
config:
  dataset: '{data_root}/earvn'
  mode: native-lr
  scale: 4
  protocol: matched
  sr:
    - name: realesrgan
      weights: '{weights_root}/RealESRGAN_x4plus.pth'
    - name: swinir_classical
      weights: '{weights_root}/001_classicalSR_DIV2K_s48w8_SwinIR-M_x4.pth'
    - name: swinir_realworld
      weights: '{weights_root}/003_realSR_BSRGAN_DFO_s64w8_SwinIR-M_x4_GAN.pth'
    - name: span
      weights: '{weights_root}/spanx4_ch48.pth'
  recognizer:
    backbones:
      - resnet18
    seeds:
      - 0
      - 1
      - 2
```

Appendix B. Console session

Listing 3 is a session on the reference machine (NVIDIA GeForce RTX 3080). `init` writes a configuration and refuses `run` until each `[CHECK]` marker is removed. `sr4rec reproduce quickstart` then reports that the quickstart expected results are not frozen in this development version and writes the demo configuration for `run`.

Listing 3: Console session: `init`, `run --dry-run`, `run` and `reproduce` on the Oxford-IIIT Pet quickstart.

```
$ sr4rec init examples/data/pets_mini --output appendix_b.yaml --force
Dataset OK: 300 images, 10 classes; labels.csv has a split column.
Image short side: min 256 px, median 256 px, max 256 px.
```

```

Default split (column): train 180 / val 20 / test 100 images.
Wrote appendix_b.yaml
Next: confirm 'mode' (marked [CHECK]), add your SR models under 'sr:', then run
sr4rec run appendix_b.yaml --dry-run

$ sr4rec run appendix_b.yaml --dry-run
Error: The configuration still contains [CHECK] markers. Confirm or edit each value, then
delete the "[CHECK]" marker:
  line 36: mode: synthetic # [CHECK] suggested from the median short side 256 px (>= 112
px); confirm, then delete this marker

$ sed -i 's/ # \[CHECK\].*//' appendix_b.yaml

$ sr4rec run appendix_b.yaml --dry-run
Config          OK
Dataset          OK (300 images, 10 classes)
Split            OK train 180 / val 20 / test 100 images
SR models
  bicubic        OK x4 built-in baseline
Plan            protocol matched: 1 image pipelines (bicubic + 0 SR) + HR, 3 backbone
(s), 3 seed(s)
               = 18 recognizer trainings (18 not cached), 0 SR images to compute
Estimated time  about 3 min on NVIDIA GeForce RTX 3080 (from a timed mini-batch)
Dry run passed. Start the real run with: sr4rec run appendix_b.yaml

$ sr4rec run runs/quickstart_config.yaml --dry-run
Config          OK
Dataset          OK (300 images, 10 classes)
Split            OK train 180 / val 20 / test 100 images
SR models
  bicubic        OK x4 built-in baseline
  tiny_espcn     OK x4 module examples/sr_models/tiny_espcn.py:TinyESPCN
Plan            protocol matched: 2 image pipelines (bicubic + 1 SR) + HR, 1 backbone
(s), 1 seed(s)
               = 3 recognizer trainings (0 not cached), 0 SR images to compute
Estimated time  about 1 s on NVIDIA GeForce RTX 3080 (from a timed mini-batch)
Dry run passed. Start the real run with: sr4rec run runs/quickstart_config.yaml

$ sr4rec run runs/quickstart_config.yaml
SR4Rec 0.1.0.dev0: run folder runs/pets_mini_2026-09-24_1109
Measuring CPU latency
Done in 8 s. Answer: No SR model changed Rank-1 accuracy significantly on any backbone.
ResNet-18: best SR tiny_espcn, -2.0 pp vs bicubic, • no difference
Report: runs/pets_mini_2026-09-24_1109/report.md

$ sr4rec reproduce quickstart
Error: The expected results of demo 'quickstart' have not been frozen in this version of
SR4Rec, so there is nothing to compare with. The demo configuration was written to
runs/quickstart_config.yaml; run it with 'sr4rec run runs/quickstart_config.yaml'.

```

Appendix C. Correctness checks and detailed results

Table C.1 lists the checks of SR4Rec against reference implementations; the test suite runs the statistical and metric checks on every change. Ta-

ble C.2 gives the D1 results per size bin.

Table C.1: Correctness checks of SR4Rec against reference implementations.

Component	Reference	Criterion	Result
SR loading and PSNR/SSIM (spandrel, Y channel, border 4)	published x4 results of SwinIR (DIV2K) and SPAN on Set5 and Set14	PSNR within 0.05 dB	SwinIR-M x4 Set5 32.75 dB / 0.9021 (published 32.72 / 0.9021); SPAN x4 Set5 32.20 / 0.8953 (published 32.20 / 0.8953); SwinIR-M x4 Set14 28.94 / 0.7914 (published 28.94 / 0.7914); SPAN x4 Set14 28.64 / 0.7828 (published 28.66 / 0.7834)
MATLAB-style bicubic	BasicSR port of MATLAB imresize	at most 1 grey level after uint8 rounding	identical to 1e-5 (BasicSR reference)
Bootstrap CI, permutation p, Holm	scipy.stats bootstrap and permutation_test; statsmodels Holm	CI and p within 0.01; Holm within 1e-10	passed (test suite)
Rank-k, macro precision, recall, F1	hand-computed examples; scikit-learn (zero_division = 0)	identical values	passed (test suite)
Oracle SR (HR images given as SR output)	HR upper bound of the same run	identical predictions	passed (test suite)

Table C.2: D1: Rank-1 difference with bicubic (pp) and verdict by short side of the LR image; bins with fewer than 50 test images are not tested and show their size.

SR	<32 px	32–63 px	64–127 px	≥128 px
realesrgan	n.s. −0.4	gain +3.0	n.s. +1.4	n.s. −0.3
swinir_classical	harm −5.5	harm −2.2	n.s. −0.7	n.s. −0.7
swinir_realworld	n.s. −0.1	gain +2.5	n.s. +0.8	n.s. +0.2
span	harm −5.0	n.s. −1.0	n.s. −0.6	n.s. +0.3

References

- [1] K. Grm, W. J. Scheirer, V. Štruc, Face hallucination using cascaded super-resolution and identity priors, *IEEE Transactions on Image Processing* 29 (2020) 2150–2165. doi:10.1109/TIP.2019.2945835.
- [2] M. M. Zarachoff, A. Sheikh-Akbari, D. Monekosso, Application of single image super-resolution in human ear recognition using eigenvalues, in: *Proceedings of the IEEE International Conference on Imaging Systems and Techniques (IST)*, 2018, pp. 1–6. doi:10.1109/IST.2018.8577134.
- [3] L. Markičević, P. Peer, Ž. Emeršič, Improving ear recognition with super-resolution, in: *Proceedings of the International Conference on Systems, Signals and Image Processing (IWSSIP)*, 2023, pp. 1–5. doi:10.1109/IWSSIP58668.2023.10180250.
- [4] J. Liang, J. Cao, G. Sun, K. Zhang, L. Van Gool, R. Timofte, SwinIR: Image restoration using Swin transformer, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 2021, pp. 1833–1844.
- [5] X. Wang, L. Xie, C. Dong, Y. Shan, Real-ESRGAN: Training real-world blind super-resolution with pure synthetic data, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 2021, pp. 1905–1914.
- [6] C. Wan, H. Yu, Z. Li, Y. Chen, Y. Zou, Y. Liu, X. Yin, K. Zuo, Swift parameter-free attention network for efficient super-resolution, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2024, pp. 6246–6256. doi:10.1109/CVPRW63382.2024.00628.
- [7] Z. Wang, A. C. Bovik, H. R. Sheikh, E. P. Simoncelli, Image quality assessment: From error visibility to structural similarity, *IEEE Transactions on Image Processing* 13 (4) (2004) 600–612.
- [8] C. Fookes, F. Lin, V. Chandran, S. Sridharan, Evaluation of image resolution and super-resolution on face recognition performance, *Journal of Visual Communication and Image Representation* 23 (1) (2012) 75–93. doi:10.1016/j.jvcir.2011.06.004.

- [9] X. Wang, L. Xie, K. Yu, K. C. K. Chan, C. C. Loy, C. Dong, BasicSR: Open source image and video restoration toolbox, <https://github.com/XPixelGroup/BasicSR> (2022).
- [10] K. Zhang, KAIR: Image restoration toolbox, <https://github.com/cszn/KAIR> (2021).
- [11] chaiNNer-org, spandrel: A library for loading and running pre-trained PyTorch image models, <https://github.com/chaiNNer-org/spandrel>, version 0.4 (2024).
- [12] C. Chen, J. Mo, IQA-PyTorch: PyTorch toolbox for image quality assessment, <https://github.com/chaofengc/IQA-PyTorch> (2022).
- [13] R. Wightman, PyTorch image models, <https://github.com/huggingface/pytorch-image-models> (2019). doi:10.5281/zenodo.4414861.
- [14] C. Aybar, D. Montero, S. Donike, F. Kalaitzis, L. Gómez-Chova, A comprehensive benchmark for optical remote sensing image super-resolution, *IEEE Geoscience and Remote Sensing Letters* 21 (2024) 1–5. doi:10.1109/LGRS.2024.3401394.
- [15] A. Paszke, et al., PyTorch: An imperative style, high-performance deep learning library, in: *Advances in Neural Information Processing Systems* (NeurIPS), Vol. 32, 2019.
- [16] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei, ImageNet: A large-scale hierarchical image database, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.
- [17] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [18] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, H. Adam, Searching for MobileNetV3, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 1314–1324.

- [19] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, S. Xie, A ConvNet for the 2020s, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 11976–11986.
- [20] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, New York, 1993.
- [21] P. I. Good, Permutation, Parametric, and Bootstrap Tests of Hypotheses, 3rd Edition, Springer, New York, 2005.
- [22] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.
- [23] O. M. Parkhi, A. Vedaldi, A. Zisserman, C. V. Jawahar, Cats and dogs, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2012, pp. 3498–3505.
- [24] K. Zhang, J. Liang, L. Van Gool, R. Timofte, Designing a practical degradation model for deep blind image super-resolution, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2021, pp. 4791–4800.
- [25] V. T. Hoang, EarVN1.0: A new large-scale ear images dataset in the wild, Data in Brief 27 (2019) 104630. doi:10.1016/j.dib.2019.104630.
- [26] G. B. Huang, M. Ramesh, T. Berg, E. Learned-Miller, Labeled faces in the wild: A database for studying face recognition in unconstrained environments, Tech. Rep. 07-49, University of Massachusetts, Amherst (2007).
- [27] C. Wah, S. Branson, P. Welinder, P. Perona, S. Belongie, The Caltech-UCSD Birds-200-2011 dataset, Tech. Rep. CNS-TR-2011-001, California Institute of Technology (2011).